

Guía de Evolución Arquitectónica: De Monolitos MVC a APIs RESTful Seguras con PHP y MySQL

1 El Puente de Transición: De Vistas Monolíticas Acopladas a la Arquitectura de Servicios

En el ciclo académico de Desarrollo de Aplicaciones Web II, se ha implementado una base sólida guiada por el patrón de diseño clásico Modelo-Vista-Controlador (MVC), empleando PHP nativo y MySQL para la persistencia de datos. En dicha arquitectura, la interacción del sistema dependía de un despachador frontal unificado (`index.php`) encargado de enrutar las peticiones, instanciar controladores de presentación y renderizar de forma directa plantillas HTML estructuradas en layouts (`header.php`, `navbar.php` y `footer.php`).

Incluso los mecanismos de interacción dinámica basados en **Comunicación Asíncrona, AJAX, Fetch API y Promises** que se introdujeron anteriormente en el curso —como el listado asíncrono implementado en `listado_async.php` y su respectivo script de control `listado_async.js`— seguían operando bajo un entorno de acoplamiento lógico, donde el cliente web dependía de una estructura de respuestas prediseñadas o formateadas de manera exclusiva para la interfaz del navegador.

El desarrollo de software a gran escala y la arquitectura de sistemas modernos exigen una migración conceptual hacia la separación absoluta de responsabilidades mediante la creación de APIs (de Toledo et al., 2021). En lugar de que el backend asuma el costo computacional de renderizar interfaces gráficas y empaquetar maquetación visual, el servidor se transforma en un proveedor puro de servicios que expone el estado y los datos del sistema en formatos serializados ligeros como JSON (Fielding, 2000).

Este desacoplamiento estratégico permite que una única base de datos y lógica de negocio sirvan simultáneamente a múltiples clientes heterogéneos: aplicaciones web de una sola página (SPAs en React, Vue o Angular), aplicaciones móviles nativas (Android/iOS) o integraciones con microservicios externos de terceros (Kotstein et al., 2023).

2 Principios Fundamentales de la Arquitectura REST y Semántica HTTP

La transferencia de estado representacional (REST) es un estilo de arquitectura definido por el Dr. Roy Fielding en su tesis doctoral para guiar el diseño y evolución de sistemas hipermedia distribuidos (Fielding, 2000). REST no es un protocolo ni un lenguaje de programación, sino una filosofía compuesta por seis restricciones clave que garantizan la escalabilidad, la fiabilidad y el rendimiento óptimo de las aplicaciones web (Fielding, 2000):

1. **Cliente-Servidor:** Separación clara de la interfaz de usuario y la persistencia de datos.

Dimensión de Análisis	Esquema Monolítico (MVC Tradicional)	Arquitectura de Servicios (API REST)
Responsabilidad del Servidor	Procesa lógica de negocio, realiza consultas de datos y renderiza la interfaz HTML final.	Procesa reglas de negocio, persiste datos y retorna representaciones estructuradas en JSON.
Formato de Transferencia	Código HTML dinámico acoplado con scripts y layouts de servidor.	Documentos serializados ligeros sin información de presentación visual (JSON).
Acoplamiento Visual	Alto; cualquier alteración estética exige modificar archivos del servidor backend.	Nulo; los clientes consumen los endpoints de datos y aplican sus propios estilos.
Consumo Multicliente	Complejo y redundante; requiere duplicar controladores para dar soporte a apps móviles.	Nativo; una única API sirve de forma homogénea a navegadores, dispositivos móviles y sistemas externos.
Gestión de Sesiones	Con estado (Stateful) mediante variables de sesión físicas (\$_SESSION).	Sin estado (Stateless); la autorización viaja en cada solicitud HTTP de manera aislada.

Cuadro 1: Comparación estructural entre el patrón monolítico y la arquitectura orientada a servicios.

2. **Sin Estado (Statelessness):** Cada petición del cliente debe contener toda la información requerida por el servidor para comprenderla y procesarla, sin depender de un contexto guardado en el servidor web.
3. **Caché:** Las respuestas deben catalogarse de manera explícita como almacenables o no en caché para optimizar la latencia del canal de red.
4. **Sistema en Capas:** El cliente no puede saber con certeza si está conectado de forma directa al servidor de destino o a un intermediario (balanceador de carga, proxy de seguridad).
5. **Código bajo Demanda (Opcional):** Envío de scripts ejecutables al cliente para aumentar dinámicamente su funcionalidad.
6. **Interfaz Uniforme:** Contrato de diseño estandarizado basado en la identificación de recursos a través de URIs unificadas, manipulación a través de representaciones (JSON), uso de mensajes autodescriptivos (cabeceras HTTP) y el uso de hipermédios (HATEOAS).

Para implementar de manera práctica una interfaz uniforme en un backend PHP, debemos utilizar de forma semántica las operaciones del protocolo HTTP. Los verbos HTTP no actúan como simples comandos de transmisión, sino como operadores de estado lógicos aplicados directamente sobre los recursos (Kotstein et al., 2023).

Estudios contemporáneos evidencian que ignorar la semántica estándar de HTTP o tergiversar el propósito de los verbos y códigos de estado degrada de manera sustancial la Experiencia del Desarrollador (DX), incrementando significativamente la propensión a introducir fallas graves durante la fase de integración (Kotstein et al., 2023).

Método HTTP	Acción CRUD	Seguro	Idempotente	Códigos de Estado HTTP y Significado Semántico
GET	Leer / Ob- tener	Sí	Sí	200 OK (Éxito con datos); 404 Not Found (Inexistencia del recurso solicitado).
POST	Crear / In- sertar	No	No	201 Created (Recurso creado); 400 Bad Request (Errores en datos de validación).
PUT	Actualizar completo	No	Sí	200 OK (Modificado con respuesta); 204 No Content (Éxito sin cuerpo de retorno).
PATCH	Actualizar parcial	No	No	200 OK (Modificación parcial exitosa); 422 Unprocessable Entity (Error de lógica).
DELETE	Eliminar	No	Sí	200 OK (Eliminación exitosa con detalles); 204 No Content (Borrado sin cuerpo de retorno).

Cuadro 2: Mapeo semántico de operaciones y propiedades de los métodos del protocolo HTTP.

3 Refactorización del Núcleo MVC PHP para la Exposición de Servicios

En nuestro framework MVC original, el controlador base dependía del motor de vistas HTML (Views.php). Para adaptarlo al paradigma RESTful, crearemos un controlador base de servicios denominado ApiController.

Esta clase interceptará las peticiones del cliente, leerá flujos de datos JSON directamente de la entrada estándar del servidor web (php://input) y proporcionará métodos automatizados para el retorno de datos estructurados con cabeceras de respuesta precisas.

```

1 <?php
2 // Libraries/Core/ApiController.php
3 // Clase base para controladores RESTful. Reemplaza el renderizado de vistas por
  respuestas estructuradas en JSON.
4
5 abstract class ApiController {
6     protected $model;
7     protected array $requestHeaders = [];
8     protected string $requestMethod;
9     protected ?array $requestBody = null;
10
11     public function __construct() {
12         $this->requestMethod = $_SERVER['REQUEST_METHOD'];
13         $this->requestHeaders = getallheaders();
14         $this->parseIncomingRequest();
15         $this->loadModel();
16     }
17
18     private function loadModel(): void {
19         // Enlace dinámico con los modelos del sistema base
20         $modelName = str_replace("ApiController", "Model", get_class($this));
21         $modelPath = "Models/{$modelName}.php";
22         if (file_exists($modelPath)) {
23             require_once $modelPath;
24             $this->model = new $modelName();
25         }
26     }
27

```

```

28 private function parseIncomingRequest(): void {
29     // Para metodos que alteran el estado del servidor, procesamos el cuerpo
    de la peticion
30     if (in_array($this->requestMethod, ['POST', 'PUT', 'PATCH', 'DELETE']))
    {
31         $contentType = $this->requestHeaders['Content-Type'] ?? $this->
    requestHeaders['content-type'] ?? '';
32
33         if (str_contains($contentType, 'application/json')) {
34             // Lectura segura del flujo crudo de entrada de PHP
35             $rawInput = file_get_contents('php://input');
36             $decoded = json_decode($rawInput, true);
37
38             if (json_last_error() === JSON_ERROR_NONE) {
39                 $this->requestBody = $decoded;
40             } else {
41                 $this->sendJsonResponse(['error' => 'La sintaxis JSON de la
    peticion es invalida'], 400);
42             }
43             } else if ($this->requestMethod !== 'DELETE') {
44                 // Forzamos el uso de application/json en metodos mutacionales
45                 $this->sendJsonResponse(['error' => 'El tipo de contenido debe
    ser application/json'], 415);
46             }
47         }
48     }
49
50     protected function sendJsonResponse(array $data, int $statusCode = 200):
    void {
51         // Limpiamos buffers abiertos de salida previos para evitar fugas de
    advertencias PHP en el JSON
52         if (ob_get_length() > 0) {
53             ob_clean();
54         }
55         header('Content-Type: application/json; charset=utf-8');
56         http_response_code($statusCode);
57         echo json_encode($data, JSON_UNESCAPED_UNICODE | JSON_PRETTY_PRINT);
58         exit();
59     }
60 }

```

Listing 1: Implementación de la clase core ApiController en PHP.

A continuación, crearemos un controlador especializado en la entidad de negocio **Productos**. Este controlador mapea dinámicamente las solicitudes del cliente y ejecuta las operaciones de persistencia consumiendo directamente los métodos del Query Builder de la clase base Model.php del framework (all(), find(), insert(), update() y delete()).

```

1 <?php
2 // Controllers/ProductosApiController.php
3 // Controlador RESTful encargado de exponer las operaciones CRUD para el recurso
    Productos.
4
5 class ProductosApiController extends ApiController {
6
7     public function index(?string $params = ""): void {
8         // Despachador de acciones basado en el metodo de la solicitud HTTP
9         switch ($this->requestMethod) {
10             case 'GET':

```

```
11         $this->getProducts($params);
12         break;
13     case 'POST':
14         $this->createProduct();
15         break;
16     case 'PUT':
17         $this->updateProduct($params);
18         break;
19     case 'DELETE':
20         $this->deleteProduct($params);
21         break;
22     default:
23         header("Allow: GET, POST, PUT, DELETE");
24         $this->sendJsonResponse(['error' => 'Metodo HTTP no permitido en
este recurso'], 405);
25         break;
26     }
27 }
28
29 private function getProducts(?string $id): void {
30     if (!empty($id)) {
31         $idVal = intval($id);
32         $producto = $this->model->find($idVal);
33
34         if (!$producto) {
35             $this->sendJsonResponse(['error' => 'El producto solicitado no
existe'], 404);
36         }
37         $this->sendJsonResponse(['status' => true, 'data' => $producto]);
38     } else {
39         $productos = $this->model->all();
40         $this->sendJsonResponse(['status' => true, 'data' => $productos]);
41     }
42 }
43
44 private function createProduct(): void {
45     $data = $this->requestBody;
46
47     if (empty($data['nombre']) || empty($data['precio']) || empty($data['sku
'])) {
48         $this->sendJsonResponse(['error' => 'Faltan campos obligatorios para
el registro'], 400);
49     }
50
51     $nuevoProducto = [
52         'nombre' => htmlspecialchars(strip_tags($data['nombre'])),
53         'precio' => floatval($data['precio']),
54         'sku' => htmlspecialchars(strip_tags($data['sku'])),
55         'creado_at' => date('Y-m-d H:i:s')
56     ];
57
58     $exito = $this->model->insert($nuevoProducto);
59
60     if ($exito) {
61         $this->sendJsonResponse([
62             'status' => true,
63             'message' => 'Producto registrado exitosamente'
64         ], 201);
65     }
```

```
65     } else {
66         $this->sendJsonResponse(['error' => 'Ocurrio un error interno en el
servidor'], 500);
67     }
68 }
69
70 private function updateProduct(?string $id): void {
71     if (empty($id)) {
72         $this->sendJsonResponse(['error' => 'El identificador del producto
es requerido'], 400);
73     }
74
75     $idVal = intval($id);
76     $data = $this->requestBody;
77
78     $productoExistente = $this->model->find($idVal);
79     if (!$productoExistente) {
80         $this->sendJsonResponse(['error' => 'El recurso a modificar no
existe'], 404);
81     }
82
83     $datosActualizar = [];
84     if (isset($data['nombre'])) $datosActualizar['nombre'] =
htmlspecialchars(strip_tags($data['nombre']));
85     if (isset($data['precio'])) $datosActualizar['precio'] = floatval($data[
'precio']);
86     if (isset($data['sku'])) $datosActualizar['sku'] = htmlspecialchars(
strip_tags($data['sku']));
87
88     if (empty($datosActualizar)) {
89         $this->sendJsonResponse(['error' => 'No se proporcionaron datos para
actualizar'], 400);
90     }
91
92     $exito = $this->model->update($idVal, $datosActualizar);
93
94     if ($exito) {
95         $this->sendJsonResponse(['status' => true, 'message' => 'Producto
actualizado de forma exitosa']);
96     } else {
97         $this->sendJsonResponse(['error' => 'No se pudo actualizar el
registro en el servidor'], 500);
98     }
99 }
100
101 private function deleteProduct(?string $id): void {
102     if (empty($id)) {
103         $this->sendJsonResponse(['error' => 'El identificador es requerido'
], 400);
104     }
105
106     $idVal = intval($id);
107     $productoExistente = $this->model->find($idVal);
108
109     if (!$productoExistente) {
110         $this->sendJsonResponse(['error' => 'El recurso no fue encontrado'],
404);
111     }
112 }
```

```
112     $exito = $this->model->delete($idVal);
113
114     if ($exito) {
115         $this->sendJsonResponse(['status' => true, 'message' => 'Producto
116 eliminado correctamente']);
117     } else {
118         $this->sendJsonResponse(['error' => 'Fallo la eliminacion interna
119 del recurso'], 500);
120     }
121 }
```

Listing 2: Controlador de API de Productos con enrutamiento de verbos HTTP.

4 Análisis de Errores Comunes de Diseño y Acumulación de Deuda Técnica

En el ciclo académico y profesional, los ingenieros de software en formación tienden a enfocarse de manera exclusiva en las metas funcionales del código (hacer que el sistema “funcione”), descuidando por completo las restricciones del diseño arquitectónico y las convenciones internacionales (Di Meglio et al., 2025).

Este descuido sistemático genera un incremento desmedido de la **Deuda Técnica Arquitectónica (DTA)** (de Toledo et al., 2021). La DTA representa los costos acumulados por decisiones de desarrollo rápidas y deficientes en etapas iniciales que, de no corregirse a tiempo, penalizarán de forma drástica los futuros sprints de desarrollo (Fasolino et al., 2026). La acumulación de deuda técnica actúa bajo una función de interés compuesto de índole financiero, donde cada retraso o parche eleva de manera exponencial el coste de mantenimiento final (de Toledo et al., 2021):

$$I_{\text{acumulado}} = P(1 + r)^t \quad (1)$$

Donde P es el esfuerzo básico inicial de refactorización requerido para sanear el diseño de la API a un estándar aceptable, r representa la tasa de fricción o penalización del software (el retraso adicional acumulado en el desarrollo de nuevas funciones debido al código deficiente existente) y t el tiempo de operación transcurrido en el proyecto web.

El análisis de miles de repositorios y proyectos mediante escáneres estáticos automatizados como RESTruler demuestra que las violaciones de diseño más recurrentes se agrupan en categorías específicas detalladas en la Tabla 3.

No corregir estas inconsistencias genera el anti-patrón de “Servicios Acoplados” o “Base de Datos Compartida” (de Toledo et al., 2021). Esto ocurre cuando los desarrolladores, al no contar con un contrato unificado o APIs estables, deciden eludir las capas de servicio web e integrar directamente sus clientes consumiendo las tablas de la base de datos MySQL, provocando que un simple cambio estructural rompa en cascada todas las aplicaciones satélite en producción de manera irreversible.

5 Seguridad en Endpoints: Mitigación de Vulnerabilidades Críticas (OWASP API Top 10)

Al migrar a un enfoque de servicios web, la interfaz gráfica (HTML/CSS) deja de ser la primera línea de contención. Los atacantes y herramientas automatizadas ya no interactúan con los

Infracción del Diseño REST	Identificador RESTRu-ler	Pre-sencia Prome-dio	Refactorización de Endpoint Recomenda-da
Uso de guiones bajos o camelCase en URIs	Hyphens	98%	GET /api/obtenerProductos → GET /api/v1/productos
Uso de sustantivos en singular	PluralNoun	88%	GET /api/v1/producto/15 → GET /api/v1/productos/15
Túneles de acción en rutas GET (Modificación)	NoTunnel	83%	GET /api/productos/delete?id=15 → DELETE /api/v1/productos/15
Uso de letras mayúsculas en rutas	Lowercase	Alta	GET /api/Mis-Notas → GET /api/mis-notas
Ausencia de cabecera Content-Type estándar	ContentType	Alta	Configurar la cabecera obligatoria: Content-Type: application/json.

Cuadro 3: Violaciones de diseño en APIs documentadas según análisis de código estático (adaptado de Di Meglio et al., 2025).

botones de un sitio web, sino que dirigen peticiones HTTP construidas a mano directo hacia el puerto de escucha del backend (OWASP, 2023).

La amenaza más severa para los servicios backend, catalogada de forma histórica en el primer puesto de la lista global OWASP API Security Top 10, es la **Autorización Fallida a Nivel de Objeto (BOLA - Broken Object Level Authorization, conocida anteriormente como IDOR)** (OWASP, 2023). BOLA se suscita cuando un servicio expone endpoints que permiten identificar recursos directamente (como GET /api/v1/notas/12), y el backend recupera el registro de la base de datos de MySQL sin validar previamente si el usuario actualmente autenticado posee la pertenencia o los permisos legales correspondientes sobre dicho objeto específico (OWASP, 2023).

Para mitigar BOLA, los desarrolladores deben adherirse a dos principios inquebrantables de codificación:

- Validación Server-Side Estricta:** Bajo ninguna premisa se debe asumir la identidad o derechos de un cliente a partir de variables de entorno del navegador o parámetros editables en la URL.
- Consultas Vinculadas al Inquilino (Tenant-Aware Queries):** Toda consulta dirigida a la base de datos MySQL empleando el Query Builder debe incorporar filtros condicionales estrictos que involucren el identificador único del usuario verificado (user_id) mediante cláusulas WHERE.

Para entender el riesgo inherente de este ataque en nuestra arquitectura PHP MVC, se expone a continuación un controlador didáctico con una falla BOLA crítica y su subsiguiente refactorización defensiva segura empleando los métodos selectivos de Model.php.

```
1 <?php
2 // Controllers/NotaVulnerableApiController.php
3 // Demostracion practica de endpoint inseguro. Permite explotacion IDOR/BOLA de
  forma directa.
4
5 class NotaVulnerableApiController extends ApiController {
6
7     public function index(?string $id = ""): void {
8         if ($this->requestMethod !== 'GET') {
```



```

9         $this->sendJsonResponse(['error' => 'Metodo no permitido'], 405);
10     }
11
12     if (empty($id)) {
13         $this->sendJsonResponse(['error' => 'El ID de la nota es requerido'
14     ], 400);
15     }
16
17     $notaId = intval($id);
18
19     // FALLA CRITICA: Se recupera la nota de MySQL consultando solo el ID de
20     registro.
21     // Un usuario malicioso autenticado con el token del Usuario 5 puede
22     realizar peticiones a
23     // /nota-vulnerable-api/index/104 y leer la nota privada perteneciente
24     al Usuario 8.
25     $nota = $this->model->find($notaId);
26
27     if (!$nota) {
28         $this->sendJsonResponse(['error' => 'Nota no encontrada'], 404);
29     }
30
31     $this->sendJsonResponse(['status' => true, 'data' => $nota]);
32 }

```

Listing 3: Implementación de endpoint vulnerable a ataques de tipo BOLA.

A continuación, implementamos una estrategia de defensa profunda sobre el mismo endpoint. Para ello, requerimos la presencia de una cabecera segura de autenticación (X-API-KEY) y reestructuramos la lógica de persistencia limitando la extracción de datos al inquilino validado en el backend de forma segura.

```

1 <?php
2 // Controllers/NotaSeguraApiController.php
3 // Controlador seguro que autentica mediante API Key y acota la consulta SQL al
4 // propietario de los datos.
5
6 class NotaSeguraApiController extends ApiController {
7     private int $authenticatedUserId;
8
9     public function __construct() {
10         parent::__construct();
11         $this->enforceBasicAuthentication();
12     }
13
14     private function enforceBasicAuthentication(): void {
15         // Intercepcion de la cabecera personalizada de autorizacion
16         $apiKey = $this->requestHeaders['X-API-Key'] ?? $this->requestHeaders['x
17         -api-key'] ?? null;
18
19         if (!$apiKey) {
20             $this->sendJsonResponse(['error' => 'Se requiere cabecera de
21             autorizacion X-API-Key'], 401);
22         }
23
24         $userId = $this->authenticateApiKey($apiKey);
25
26         if (!$userId) {

```

```
24     $this->sendJsonResponse(['error' => 'La credencial provista es
invalida o expiro'], 403);
25     }
26
27     // Persistimos la identidad del inquilino validado para usarla en las
consultas del controlador
28     $this->authenticatedUserId = $userId;
29     }
30
31     private function authenticateApiKey(string $key): ?int {
32         // En un entorno de produccion, esto validaria una firma criptografica o
realizaria una consulta en base de datos.
33         // Datos para desarrollo academico universitario:
34         $mockApiKeys = [
35             'key_estudiante_epis_daw2_user10' => 10, // Pertenece al Usuario ID:
10
36             'key_estudiante_epis_daw2_user11' => 11 // Pertenece al Usuario ID:
11
37         ];
38         return $mockApiKeys[$key] ?? null;
39     }
40
41     public function index(?string $id = ""): void {
42         if ($this->requestMethod !== 'GET') {
43             $this->sendJsonResponse(['error' => 'Metodo HTTP no permitido'],
405);
44         }
45
46         if (empty($id)) {
47             $this->sendJsonResponse(['error' => 'El ID es obligatorio'], 400);
48         }
49
50         $notaId = intval($id);
51
52         // SOLUCION DEFENSIVA BOLA: Reescritura del Query Builder del framework
base
53         // Se fuerza que el registro coincida con el ID solicitado y que el
propietario sea el usuario autenticado
54         $nota = $this->model
55             ->select("id, titulo, contenido, user_id")
56             ->where([
57                 "id = {$notaId}",
58                 "user_id = {$this->authenticatedUserId}"
59             ])
60             ->first();
61
62         // Si el registro pertenece a otro usuario o no existe en la base de
datos, retornamos un 404 generico.
63         // Esto evita el descubrimiento y enumeracion de IDs secuenciales por
fuerza bruta por parte de atacantes.
64         if (!$nota) {
65             $this->sendJsonResponse(['error' => 'El recurso no existe o no tiene
los privilegios de autorizacion requeridos'], 404);
66         }
67
68         $this->sendJsonResponse([
69             'status' => true,
70             'data' => $nota
71         ];
```

```
71     ] );  
72   }  
73 }
```

Listing 4: Mitigación y diseño defensivo contra vulnerabilidades BOLA.

6 Estrategias Pedagógicas: Fomento del Debate y Pensamiento Crítico

El aprendizaje profundo de la ingeniería de software exige que los estudiantes cuestionen las decisiones técnicas y entiendan las repercusiones a largo plazo de un diseño arquitectónico deficiente. Se plantean tres escenarios prácticos de debate grupal e individual para las sesiones de clase:

6.1 Preguntas Dinamizadoras y Casos de Estudio

- **Escenario de Debate 1 (Contratos de API vs. Desarrollo "On-The-Fly"):** Un equipo decide prescindir de la fase de planificación de la API y de herramientas de modelado como OpenAPI/Swagger, optando por escribir código y "probar en caliente". ¿Qué consecuencias tendrá esta decisión en el desarrollo paralelo de las aplicaciones móviles y clientes SPA creados por otros miembros del equipo? ¿Cómo afectará los costos del proyecto si el backend cambia constantemente de opinión respecto a los campos que retorna en el JSON?
- **Escenario de Debate 2 (Develar la Estructura de Datos):** Si exponemos de manera directa nombres de columnas físicas de MySQL en las llaves de nuestro payload JSON (por ejemplo, "usr_fk_profile_tbl_id": 5), ¿cómo se ve afectada la capacidad de evolución del motor de bases de datos de la empresa? ¿Qué riesgos de seguridad y facilidad de ingeniería de reversa introduce esta práctica?
- **Escenario de Debate 3 (La Ilusión de la Seguridad del Cliente):** Un programador afirma: *"No me preocupo por validar el rol de administración en mi API porque la interfaz web en JavaScript oculta automáticamente el botón de eliminación a los usuarios comunes"*. ¿Qué pasaría si un atacante elude por completo el navegador e intercepta los endpoints enviando llamadas REST directamente vía Postman o cURL?

7 Conclusiones y Directrices Técnicas de Integración

La transición hacia una arquitectura orientada a servicios en PHP y MySQL exige el cumplimiento de los siguientes lineamientos y buenas prácticas profesionales:

- **Aislamiento Absoluto de Presentación:** Descartar de forma categórica el uso de fragmentos de código HTML o elementos visuales embebidos dentro de las respuestas lógicas del backend. Los endpoints deben devolver exclusivamente representaciones de datos puras y bien estructuradas en formato JSON.
- **Eliminación de la Dependencia del Estado Físico:** La restricción de *Statelessness* requiere prescindir del uso de variables de sesión locales como `$_SESSION` en PHP para los endpoints de servicios, migrando a esquemas robustos y seguros de intercambio de tokens (como API Keys o Bearer Tokens) integrados en las cabeceras HTTP de cada petición del cliente.

- **Tratamiento Semántico de Errores e Incompatibilidades:** La manipulación errónea del protocolo HTTP —como retornar códigos 200 OK acompañados de cuerpos de mensaje de error (`{"error": "Fallo la actualizacion"}`)— rompe los estándares globales de integración y dificulta el manejo de middleware automáticos de red. Se debe utilizar la gama de respuestas HTTP de forma precisa (como 400 para datos erróneos, 401 para accesos no autenticados, y 404 para recursos no hallados).
- **Diseño de Consulta Seguro Orientado al Inquilino:** Toda operación mutacional (inserción, actualización, eliminación) o de lectura que haga uso de identificadores provistos externamente en la URL o en el payload JSON del cuerpo de petición debe forzar un análisis condicional robusto en MySQL, acotando el Query Builder al ID del inquilino autenticado de forma inequívoca en el backend para repeler por completo vulnerabilidades de tipo BOLA/IDOR.

Referencias

- [1] Di Meglio, S., Pontillo, V., & Starace, L. L. L. (2025). REST in Pieces: RESTful Design Rule Violations in Student-Built Web Apps. In *Proceedings of the 51st Euromicro Conference on Software Engineering and Advanced Applications (SEAA)* (pp. 191–200).
- [2] Fasolino, A. R., De Luca, M., Perlotto, M., & Tramontana, P. (2026). Architectural Anti-Patterns in Student-Developed Microservice Architectures: An Exploratory Study. *arXiv preprint arXiv:2602.07147*.
- [3] Fielding, R. T. (2000). *Architectural styles and the design of network-based software architectures* (Doctoral dissertation, University of California, Irvine).
- [4] Kotstein, S., Bogner, J., & Pfaff, T. (2023). Do RESTful API design rules have an impact on the understandability of Web APIs? *Empirical Software Engineering*, 28(6), 142. <https://doi.org/10.1007/s10664-023-10365-z>
- [5] OWASP. (2023). *OWASP API Security Top 10 - 2023*. Open Web Application Security Project. <https://owasp.org/www-project-api-security/>
- [6] Toledo, S. S. d., Martini, A., & Sjøberg, D. I. K. (2021). Identifying architectural technical debt, principal, and interest in microservices: A multiple-case study. *Journal of Systems and Software*, 177, 110968. <https://doi.org/10.1016/j.jss.2021.110968>